

Syntax-Directed Translation, Intermediate-code Generation and Run-Time Environments

SYLLABUS

Syntax-Directed Translation: Syntax-Directed Definitions, Evaluation Orders for SDD's, Syntax-Directed Translation Schemes, Implementing L-Attributed SDD's.

Intermediate-Code Generation: Variants of Syntax Trees, Three-Address Code.

Run-Time Environments: Stack Allocation of Space, Access to Nonlocal Data on the Stack, Heap Management.

LEARNING OBJECTIVES

- ✓ Concept of Syntax-Direct Definitions and their Evaluation Order
- ✓ Use of Syntax Directed Translations
- ✓ About Syntax Directed Translation Schemes
- ✓ Process of Implementing L-Attributed SDD's
- ✓ Various Types of Syntax Trees
- ✓ About Three Address Code, its Types and Declarations
- ✓ Concept of Stack Allocation of Space
- ✓ Procedure to Access Nonlocal Data on Stack
- ✓ Concept of Heap Management.

INTRODUCTION

Syntax-Directed Translation (SDT) is a process that add semantic rules to productions of context free grammar. It defines the evaluation order of semantic rules. In SDT, the input is the input token stream and the expected output is the evaluation order of the semantic rules. Here, the input string is parsed to generate a parse tree, then by traversed. This determines the evaluation order of semantic rule for constructing a dependency graph. A syntax-directed definition is an attribute grammar in which each grammar symbol has a set of attributes (the properties of grammar symbols) and each production has a set of semantic rules.

The intermediate code generator performs a conversion of syntax tree into a code called target code. The intermediate code should be easily transferred to target code and intermediate code generation should not take much time. The intermediate code can be of three types such as postfix notation, syntax trees and three address code.

A compiler must organize the run-time memory of the target machine to represent the data objects manipulated by the source program. In run time environment, stack allocation of space can be done based on activation tree and activation record. An activation tree is used to determine the way the control flow between procedures. Whereas the activation record or frame is contiguous block of storage that contains all information required for single execution of procedure. Moreover, heap is divided into pieces so that it can be allocated to activation records or other objects.

5.1 SYNTAX - DIRECTED TRANSLATION

5.1.1 Syntax - Directed Definitions

Q1. What is syntax directed translation? What is syntax directed definition? Give its general form.

Answer :

Syntax Directed Translation (SDT)

Syntax Directed Translation (SDT) add semantic rules to productions of context free grammar. It defines the evaluation order of semantic rules and hence the abstraction at this level is less.

In SDT, the input is the input token stream and the expected output is the evaluation order of the semantic rules. Here the input string is parsed to generate a parse tree, then by traversed. This determines the evaluation order of semantic rule constructing a dependency graph, for it the parse tree can be determined later. The token stream is translated for evaluating the semantic rules. The evaluation may result in code generation, symbol table updation, error message generation and other activities.

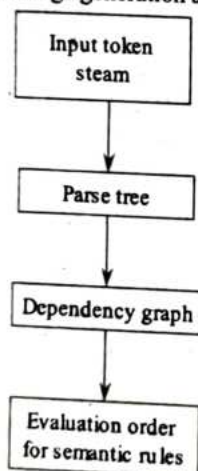


Figure: Syntax Directed Translation Procedure

Syntax Directed Definition

A syntax-directed definition is an attribute grammar in which each grammar symbol has a set of attributes (the properties of grammar symbols) and each production has a set of semantic rules. The semantic rule computes the attribute values of the grammar symbols that appear in the production. Computing the attribute values at each node in the parse tree is called *annotating* or *decorating the parse tree* and the parse tree that shows attribute values is called an *annotated parse tree*.

A syntax directed definition is a combination of a grammar and the set of semantic rules. Each nonterminal of the grammar can have two types of attributes.

1. Synthesized attributes
2. Inherited attributes.

1. Synthesized Attributes

Synthesized attributes are the attributes whose values at a node in a parse tree are computed from attribute values of its children. They have a property that their values can be evaluated during a single bottom-up transversal of the parse tree.

2. Inherited Attributes

Inherited attributes are the attributes whose values at a node in a parse tree are computed from attribute values of its parent node (LHS nonterminal) and/or its siblings (RHS nonterminals). By using inherited attributes, dependency on any context of a programming language can be shown. For example, with inherited attributes, it can be decided whether an address or the value of the identifier keeping track about its appearance either on left or right side of an assignment statement.

General Form of a Syntax-directed Definition

For each production in a grammar G , the syntax-directed definition has a semantic rule. If there is a production $B \rightarrow \theta$, the syntax directed definition has a set of semantic rules (associated with $B \rightarrow \theta$) of the form $b = \text{fun}(d_1, d_2, \dots, d_j)$.

Where,

fun is a function and

b may be,

- (i) A synthesized attribute of B , or
 - (ii) An inherited attribute of B .
- and $d_1, d_2, \dots, d_j$ are attributes for terminals and nonterminals of the grammar.

❖ The starting and terminating symbols of a grammar are assumed to have synthesized attributes.

❖ Thus, from the statement $b = \text{fun}(d_1, d_2, \dots, d_j)$ it is clear that ' b ' is dependent on $d_1, d_2, \dots, d_j$.

Q2. Describe about S-attributed and L-attributed grammars.

Answer :

S-attributed Grammar

A syntax-directed definition is said to be S-attributed if the attributes of every grammar symbol are synthesized. In S-attributed grammars, the semantic functions are defined on right hand side of the production rule in terms of symbols where as LHS of same production is assigned with those value that are returned from the function. A syntax-directed definition does not provide semantic rules for terminals (token). Therefore, they are assumed to be synthesized attributes, whose values are provided by lexical analyser.

In this definition there is an integer valued synthesized attribute called *value* for each nonterminal E , T and F and there is a semantic rule for each nonterminal. The semantic rule computes the value of attribute of a nonterminal on left hand side from the attribute values of nonterminals on right hand side of a production. The semantic rule for first production prints the value evaluated for arithmetic expression E as the output. For the synthesized attribute *lexval* associated with the token digit, the lexical analyzer provides the value for it.

The above syntax-directed definition is *S-attributed definition*, since it uses only the synthesized attribute value.

The parse tree for an arithmetic expression $4 + 3 * 2$, where n is a new line is shown in the figure (1),

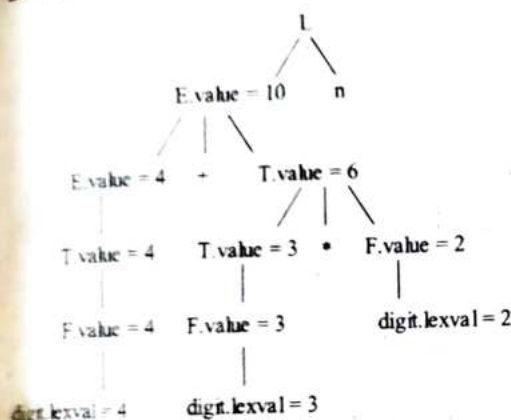


Figure (1): Annotated Parse Tree for $4 + 3 * 2$ n

The parse tree for *S-attributed* definition is annotated by computing the semantic rules for the attributes at each node in the parse tree in bottom-up fashion. For annotating, first consider the leftmost bottommost interior node which has a semantic rule as $F.value := digit.lexval$, the attribute $F.value$ gets the value 3 because its child $digit.lexval$ has 3. This value is passed to the parent of $F.value$ i.e., $T.value$, which in turn passed to $E.value$.

Now, consider the node $T.value$ having the semantic rule as $T.value := T_1.value * F.value$. The attribute value for $T.value$ is computed by multiplying the attribute values of $T_1.value$ and $F.value$ i.e., $3 * 2 = 6$. The attribute value for $E.value$ is computed by evaluating the semantic rule $E.value := E.value + T.value$, which gives $E.value := 4 + 6 = 10$. The root of the tree prints the value of the expression generated by E followed by a new line.

L-attributed Grammar

A syntax-directed definition is said to be *L-attributed* only if all the inherited attribute of X_i are in the production form $A \rightarrow x_1 x_2 \dots x_n$ where $1 \leq i \leq n$ depends only on.

- (i) The attributes of the grammar symbols $x_1, x_2, \dots, x_i$ which appears to the left of x_i in the production, and
- (ii) The inherited attributes of the nonterminal A in the left hand side of the production.

An attribute $A.S$ is said to be dependent on an attribute $B.T$ as $B.T$ was passed to the semantic function for returning a value for $A.S$.

Therefore, it can be said that every *S-attributed* definition is *L-attributed* because the above conditions apply only to inherited attributes.

Example

Consider the grammar for declarations,

$$D \rightarrow TL$$

$$T \rightarrow \text{int} \mid \text{real}$$

$$L \rightarrow L, \text{id} \mid \text{id}$$

The syntax-directed definition with inherited attribute for declarations is given below.

In this definition, the inherited attribute $L.inherit$ associated with L distributes the information about type to the various identifiers in a declaration.

Production	Semantic Rules
$D \rightarrow TL$	$L.inherit := T.type$
$T \rightarrow \text{int}$	$T.type := \text{'integer'}$
$T \rightarrow \text{real}$	$T.type := \text{'real'}$
$L \rightarrow L, \text{id}$	$L_1.inherit := L.inherit$ $addIdType(id.entry, L.inherit)$
$L \rightarrow \text{id}$	$addIdType(id.entry, L.inherit)$

Table: Syntax Directed Definition with Inherited Attribute

In this definition, there is a synthesized attribute $T.type$ associated with the nonterminal T , its value is given by the keyword in the declaration. The value of inherited attribute $L.inherit$ is set to the type in the declaration, which is then passed down in the parse tree using this attribute. The procedure $addIdType()$ enters the type associated with each identifier into the symbol table for their entry pointed to by *entry* attribute.

The nonterminal D generates a declaration which consist of keyword like *int* or *real* as well as the identifiers.

The above grammar is *L-attributed*, since the inherited attribute $L.inherit$ depends only on the attributes of the symbol T i.e. $T.type$ which appears left to it.

An annotated parse for the declaration `int id1, id2` is shown below in figure (2).

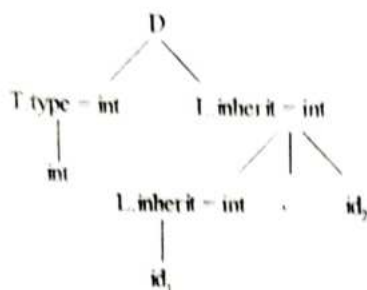


Figure (2): Annotated Parse Tree for `int id1, id2`

First the attribute value for `T.type` is evaluated from which the attribute value for `L.inherit` is evaluated in top-down fashion.

The attribute value of `L.inherit` at the two nodes in the right subtree of the root defines the type of both identifiers `id1` and `id2`. At each `L`-node the procedure `addIdType()` is called to enter the type of each identifier into the symbol table.

Q3. Write a short note on annotated parse tree.

Answer :

Annotated Parse Tree

Annotated parse tree is a specialized tree used to analyze the translation specified by SDD at each node of the parse tree. It has the ability to display attribute values of each node in parse tree. This process of generating attribute values at each level of parse tree is referred to as annotating or decorating.

Example

The annotated parse tree for given grammar is constructed in following manner.

G : $D \rightarrow TL;$
 $T \rightarrow \text{int}$
 $T \rightarrow \text{float}$
 $L \rightarrow L, id$
 $L \rightarrow id (+)$

(a) The syntax directed definition for this grammar is as follows,

Production	Semantic Rules
$D \rightarrow TL;$	$L.in := T.type$
$T \rightarrow \text{int}$	$T.type := \text{integer}$
$T \rightarrow \text{float}$	$T.type := \text{float}$
$L \rightarrow L, id$	$L.in := L.in$ $\text{Enter_type}(id.entry, L.in)$
$L \rightarrow id$	$\text{Enter_type}(id.entry, L.in)$

Figure (1): Syntax Directed Definition for the given Grammar with Inherited Attribute `L.in`

Note that `;` can be ignored by the lexical analyzer in the first production as it is a terminating symbol.

The non-terminal `D` generates a declaration which consists of a keyword either `int` or `float` followed by a list of identifiers. `'type'` is the synthesized attribute of the non-terminal `T`. For the production $D \rightarrow TL$, the semantic rule is $L.in := T.type$. This sets `L.in`, the inherited attribute to the type in the declaration. This type is inherited by other non-terminals using `L.in`. The rules for non-terminal `L` invoke a procedure called `enter-type` to add the type of each identifier to its entry in the symbol table.

(b) Given sentence is `float id1, id2, id3`

The annotated parse tree for this sentence is as follows,

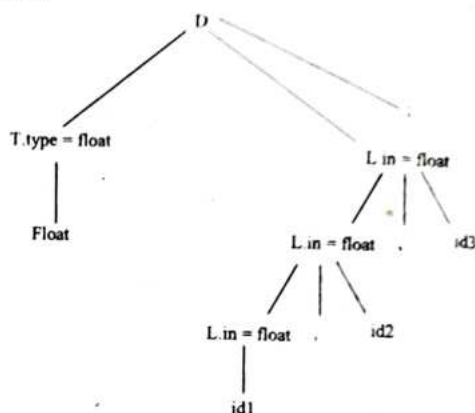


Figure (2): Annotated Parse Tree for `float id1, id2, id3`

The parse tree uses inherited attribute `'in'` at each node labeled `L`. The `L.in` value at each node gives the type of `id1`, `id2` and `id3`. To determine these values, one needs to find `T.type` value at the left child of the root and evaluate `L.in` from top to bottom at the three nodes defined by `L` in the right subtree of the root.

At every `L`-node, a procedure `enter_type` is called to insert the data type (i.e., `float`) of the identifier which is at the right child of this node into the symbol table.

Q4. What is attributed grammar? Give the syntax-directed definition for a simple desk-calculator.

Answer :

Attributed Grammar

A syntax-directed definition that does not include any function can have semantic rules with side-effects associated with them, then such a syntax-directed definition is called as an "attributed grammar".

To find out whether a given definition is an attribute grammar or not consider an example, where a semantic rule shows the dummy synthesized attribute of a nonterminal on the left side of the associated production and does not show the dummy attribute `'='`, then it is clear that this particular definition (syntax-directed definition) is not an attribute grammar.

Attribute grammar can be,

- (i) S-attributed grammar
- (ii) L-attributed grammar

Syntax-directed Definition for a Simple Desk Calculator

The grammar for a desk calculator is,

$$\begin{aligned} L &\rightarrow En \\ E &\rightarrow E + T \\ E &\rightarrow T \\ T &\rightarrow T * F \\ T &\rightarrow F \\ F &\rightarrow (E) \\ F &\rightarrow \text{digit} \end{aligned}$$

Where, n means a new line character in a language.

Grammar Production	Semantic Rules
$L \rightarrow En$	$L_1.\text{Value} := E.\text{Value}$
$E \rightarrow E + T$	$E.\text{Value} := E.\text{Value} + T.\text{Value}$
$E \rightarrow T$	$E.\text{Value} := T.\text{Value}$
$T \rightarrow T * F$	$T.\text{Value} := T.\text{Value} * F.\text{Value}$
$T \rightarrow F$	$T.\text{Value} := F.\text{Value}$
$F \rightarrow (E)$	$F.\text{Value} := E.\text{Value}$
$F \rightarrow \text{digit}$	$F.\text{Value} := \text{digit.Lexical Value}$

Table: Syntax Directed Definition

5.1.2 Evaluation Orders for SDD's

Q5. How to determine an evaluation order for the attribute instance in a given parse tree?

Answer :

Evaluation Order of SDD's

The evaluation order for attribute instance in a given parse tree can be determined with the help of a special type of graph called dependency graph.

Dependency Graph

Dependency graph for each node of parse tree can be created only if one attribute say ' x ' depend on one or more attributes say $y_1, y_2, \dots, y_m$ then the semantic rule for ' x ' is evaluated after the semantic rules for $y_1, y_2, \dots, y_m$ are evaluated.

To construct a dependency graph a prerequisite is to convert each semantic rule into the form,

$$x := f(y_1, y_2, \dots, y_m)$$

Where,

' f ' is a procedure for the semantic rule.

Algorithm of Construction of Dependency Graph

The algorithm for constructing a dependency graph is as follows,

Input

A parse tree and converted semantic rules.

Output

Dependency graph.

Step-1: Consider a node ' n ' in the parse tree and check if all nodes are considered. If yes, goto step 4 else, goto step 2.

Step-2: Consider an attribute ' a ' of the grammar symbol at ' n ' and check if all attributes are considered. If yes, goto step 1 else, goto step 3.

Step-3: Construct a node i.e., ' a ' is drawn in the dependency graph and specify ' a ' at this node. Goto step 2.

Step-4: Consider a node ' n ' in the parse tree and check if all nodes are considered. If yes goto step 10 else, goto step 5.

Step-5: Consider a semantic rule of the form $x := (y_1, y_2, \dots, y_m)$ for the production at ' n ' and check if all semantic rules at ' n ' are considered. If yes, goto step 4 else, goto step 6.

Step-6: Initialize k with the value of 1.

Step-7: An edge is drawn from y_k to x .

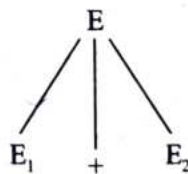
Step-8: Increment k by '1'.

Step-9: Continue step 7 till $k = m$.

Step-10: End

Example

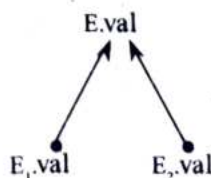
For the parse tree



And semantic rule for the production $E \rightarrow E_1 + E_2$ is,

$$E.\text{val} := f(E_1.\text{val}, E_2.\text{val})$$

Then the dependency graph using above algorithm is,



In the dependency graph, the edge from $E_1.\text{val}$ to $E.\text{val}$ shows that $E.\text{val}$ depends on $E_1.\text{val}$. Similarly, the edge from $E_2.\text{val}$ to $E.\text{val}$ shows that $E.\text{val}$ also depends on $E_2.\text{val}$.

Q6. Explain the following in detail.

- (i) S-attributed definitions
- (ii) L-attributed definitions.

Answer :

(i) S-attributed Definitions

S-Attributed Definition is a syntax directed definition that have only synthesized attributes. The evaluation of synthesized attributes in S-attributed definition is done in a single bottom-up traversal of the parse tree. S-attributed definitions are the general class of attribute grammars. It is based on LR grammar, and is implemented using LALR parser generators.

As it is known that a prefix expression has operators appearing before the operands. To get an S-attributed definition convert the grammar having infix operator to prefix operator, for doing so first convert the grammar to infix. That is left-recursion is eliminated from the grammar to get infix to prefix.

The new grammar is,

$L \rightarrow E$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid -TE' \mid E$

$E \rightarrow T$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid /FT' \mid E$

$F \rightarrow PF'$

$F \rightarrow P$

$F' \rightarrow \uparrow PF' \mid E$

$P \rightarrow (E) \mid id$

Sval and look table will be used in the S-attributed grammar. Sval is used for each non-terminal in the grammar. Sval represents the synthesized attribute, look table is a function called whenever an identifier 'id' is encountered in a production, which finds a symbol entry for id.

The S-attribute grammar is,

Grammar Productions	Semantic Rules
$L \rightarrow E$	$L.Sval := E.Sval$
$E \rightarrow TE'$	$E.Sval := T.Sval E'.Sval$
$E' \rightarrow +TE'$	$E'.Sval := '+' T.Sval E'.Sval$
$E' \rightarrow -TE'$	$E'.Sval := '-' T.Sval E'.Sval$
$E' \rightarrow E$	<i>/* No semantic action for E, as even the scanner ignores empty string */</i>
$E \rightarrow T$	$E.Sval := T.Sval$
$T \rightarrow FT'$	$T.Sval := F.Sval T'.Sval$
$T' \rightarrow *FT'$	$T'.Sval := '*' F.Sval T'.Sval$
$T' \rightarrow /FT'$	$T'.Sval := '/' F.Sval T'.Sval$
$T' \rightarrow E$	$T'.Sval := /* no semantic action */$
$T \rightarrow F$	$T.Sval := F.Sval$
$F \rightarrow PF'$	$F.Sval := P.Sval F'.Sval$
$F' \rightarrow \uparrow PF'$	$F'.Sval := '\uparrow' P.Sval F'.Sval$
$F' \rightarrow E$	<i>/* no semantic rule as an empty string means white space */</i>
$F \rightarrow P$	$F.Sval = P.Sval$
$P \rightarrow (E)$	$P.Sval = P.Sval$
$P \rightarrow id$	$P.Sval = id.looktable()$

Table: S-attribute Grammar

(B) L-attributed Definitions

An L-attributed definition is a syntax directed definition in which every inherited attribute of X_i , $1 \leq i \leq m$, on the right side of $B \rightarrow X_1 X_2 \dots X_m$ depends on,

- (i) The inherited attributes of B
- (ii) The attributes of $X_1, X_2, \dots, X_{m-1}$ to the left of X_m in the production.

Example

Consider an example of a grammar, for which an L-attributed grammar is generated and then converted into translation scheme

Let G be

$D \rightarrow TL$

$L \rightarrow L, id \mid id$

$T \rightarrow integer \mid real$

Then L-attributed grammar is,

Step 1

Grammar productions	Semantic Actions
1. $D \rightarrow TL$	$L.inherit := T.type$
2. $T \rightarrow Integer$	$T.type := 'integer'$
3. $T \rightarrow real$	$T.type := 'real'$
4. $T \rightarrow L, id$	$L.inherit := L.inherit,$ $addIdType$ $(id.entry,$ $L.inherit)$
5. $T \rightarrow id$	$L.inherit := addIdType$ $(id.entry,$ $L.inherit)$

By using the definition of L-attributed grammar

Step 2 and Step 3: L-attributed Definition's Translation Scheme

$D \rightarrow T \{L.inherit := T.type\} L$

$T \rightarrow Integer \{T.type := 'integer'\}$

$L \rightarrow real \{T.type := 'real'\}$

$L \rightarrow \{L.inherit := L.inherit\} L, id \{addIdType(id.entry, L.inherit)\}$

$L \rightarrow id \{addIdType(id.entry, L.inherit)\}$

Step 4

Deciding the evaluation order can be done by using any of the three method. However, lets use the DAG method.

- (a) Construct the syntax tree by taking only integer type declarations.

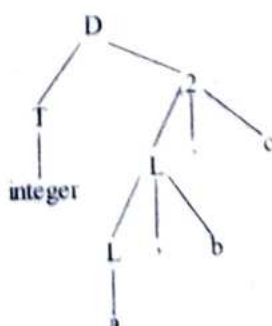


Figure (1): Syntax Tree

- (b) The above grammar is for declaration statement which can have declarations like,

integer a

integer a, b

integer a, b, c

- (c) The dependency graph or DAG gives the dependency between attributes of the nodes in a syntax tree. For the production $D \rightarrow TL$, the attribute value of D is dependent on attribute values of T and L. Thus, in a DAG an edge from T to D and L to D (as D is dependent) is given. A DAG for the above syntax tree is shown in figure (2).

(d)

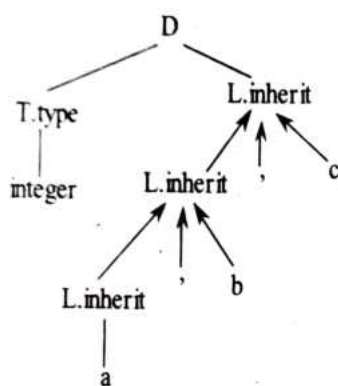


Figure (2): DAG for the Syntax Tree

Note

If a syntax directed definition has a cycle, then the SDD is called circular.

- (e) As it is discussed in step (4) point (b), that integer should come first.

∴ Give the edge $\text{integer} \rightarrow T$ in DAG '1' (label).

- (i) After an integer is generated, an identifier (at least one) is required.

∴ Give the edge $a \rightarrow L$ in DAG '3' (for the production $L \rightarrow \text{id}$, only and not $L \rightarrow L, \text{id}$).

- (ii) After data type and one identifier there can be some more identifiers that are separated by a comma.

∴ Give the edge $, \rightarrow L$ in DAG '5'.

- (iii) After a comma, there should be one more identifier.

So, $b \rightarrow L$ in DAG is given in label '6'.

Similarly, all other edges are labelled. The labels 2, 10, 4 and 7 are given only, when the labelling from all children (child) of a node is given.

2 is given from $T \rightarrow D$ only after completing

$T \rightarrow \text{integer}$, 4 from $L \rightarrow L, \text{id}$ only after finishing $L \rightarrow \text{id}$

Again 7 is from $L \rightarrow L, \text{id}$ after finishing $L \rightarrow \text{id}$ and first $L \rightarrow L, \text{id}$. Finally $L \rightarrow D$ is given 10.

∴ A DAG with evaluation order is,

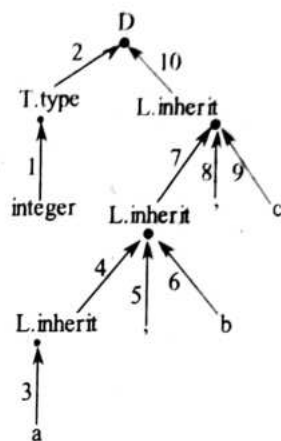


Figure (3): A DAG with Evaluation Order

5.1.3 Syntax - Directed Translation Schemes

- Q7. Explain the translation process using suitable example.

Answer :

A translation scheme is a context free grammar in which each grammar symbol has a set of attributes and the semantic actions (program fragments) enclosed between the braces {and} are embedded within the right hand side of production. The translation scheme produces an output for the given input by traversing the parse tree in depth-first order and executing the semantic actions in the order as they appear. Therefore, they are useful for specifying translation during parsing.

S-attributed Translation Scheme

S-attributed definitions have only synthesized attributes. Therefore, the translation scheme for S-attributed definitions can be constructed by creating a semantic action for each semantic rule in the definition and embedding this semantic action within the right hand side of the associated production. A semantic action consists of an assignment for each semantic rule in the definition.

Example

Consider the syntax-directed definition for desk calculator.

Production	Semantic Rules
$L \rightarrow E_n$	Print (E.value)
$E \rightarrow E_1 + T$	$E.value := E_1.value + T.value$
$E \rightarrow T$	$E.value := T.value$
$T \rightarrow T_1 * F$	$T.value := T_1.value * F.value$
$T \rightarrow F$	$T.value := F.value$
$F \rightarrow (E)$	$F.value := E.value$
$F \rightarrow \text{digit}$	$F.value := \text{digit.lexval}$

Table (a): Syntax-Directed Definition for Desk Calculator

This definition has only synthesized attribute *value* associated with all nonterminals *E*, *T* and *F*.

In the translation scheme for this definition semantic action is created for each semantic rule and attached at the end of the productions. The resulting translation scheme as shown below,

Production	Semantic Action
$L \rightarrow E_n$	{Print(E.value)}
$E \rightarrow E_1 + T$	{E.value := E ₁ .value + T.value}
$E \rightarrow T$	{E.value := T.value}
$T \rightarrow T_1 * F$	{T.value := T ₁ .value * F.value}
$T \rightarrow F$	{T.value := F.value}
$F \rightarrow (E)$	{F.value := E.value}
$F \rightarrow \text{digit}$	{F.value := digit.lexval}

Table (b): Translation Scheme for Desk Calculator

L-attributed Translation Scheme

L-attributed grammars consists of both the synthesized and inherited attributes. Therefore, the translation scheme for L-attributed definitions must satisfy the following conditions,

1. An inherited attribute for a grammar symbol on right hand sides of a production must be computed in a semantic action before the occurrence of that symbol.
2. A semantic action must not refer to the synthesized attribute of a grammar symbol that appears on its right.
3. A synthesized attribute for a nonterminal on the left hand side of a production can only be computed after computing all the attributes that it references. Such an action is usually placed at the end of the production.

Example

Consider the syntax-directed definition for declaration.

Production	Semantic Rule
$D \rightarrow TL$	$L.inherit := T.type$
$T \rightarrow \text{int}$	$T.type := \text{'integer'}$
$T \rightarrow \text{real}$	$T.type := \text{'real'}$
$L \rightarrow L_1, id$	$L.inherit := L_1.inherit$ $\text{addIdType}(id.entry, L.inherit)$
$L \rightarrow id$	$\text{addIdType}(id.entry, L.inherit)$

Table (c): Syntax-Directed Definition for Declaration Statement

This definition has both the synthesized attribute *T.type* associated with *T* and an inherited attribute *L.inherit* associated with *L*.

Following, the above requirements the translation scheme for above definition is obtained by inserting assignments corresponding to the semantic rules. The translation scheme is shown below,

Production	Semantic Action
$D \rightarrow T$	{L.inherit := T.type} L
$T \rightarrow \text{int}$	{T.type := 'integer'}
$T \rightarrow \text{real}$	{T.type := 'real'}
$L \rightarrow L_1, id$	{L ₁ .inherit := L.inherit} L ₁ {addIdType(id.entry, L.inherit)}
$L \rightarrow id$	{addIdType(id.entry, L.inherit)}

Table (d): Translation Scheme for Declaration Statement

Q8. Construct the Syntax Directed Definition to convert infix notation into postfix notation.

Answer :

The need for converting infix to postfix arises because postfix (polish) notation is used as one of the forms in which intermediate code can be generated.

Stack Implementation

The postfix expression using a stack can be explained with the help of an example.

Example

For the postfix expression $(a + b) * c$, if *a*, *b*, *c* are equivalent to 1, 3 and 5 respectively, implement the stack actions.

Proof

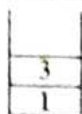
For $(a + b) * c$ the postfix notation is $ab + c *$. Given, the values of *a*, *b*, *c* are 1, 3, 5 respectively. The stack implementation is given below.

$$ab + c *$$

1. Push the first operand in the stack.



2. Push the 2nd operand in the stack.



3. Pop the two operands and look for an operator from left to right and do the appropriate operation by pushing the operator in stack.



3 is popped



1 is popped

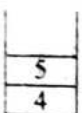


+ is pushed



3 + 1 is performed

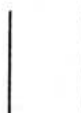
Similarly,



5 is pushed →



5 is popped →



4 is popped →



* is pushed →



5 * 4 is performed.

An expression grammar is,

$$E \rightarrow E + E \mid E - E \mid E * E \mid E \uparrow E \mid (E) \mid id$$

The SDD can be,

Grammar Productions	Semantic Rules
1. $E \rightarrow E + E$	$E_Value := E_1_Value \ E_2_Value '+'$
2. $E \rightarrow E - E$	$E_Value := E_1_Value \ E_2_Value '-'$
3. $E \rightarrow E * E$	$E_Value := E_1_Value \ E_2_Value '*'$
4. $E \rightarrow E \uparrow E$	$E_Value := E_1_Value \ E_2_Value '\uparrow'$
5. $E \rightarrow (E)$	$E_Value := E_1_Value$
6. $E \rightarrow id$	$E_Value := id$

In the SDD, the concatenation operator is used between E_1 value and E_2 value.

The SDT can be,

Grammar Productions	Semantic Rules
$E \rightarrow E + E$	{Print('+')}
$E \rightarrow E - E$	{Print('-')}
$E \rightarrow E * E$	{Print('*')}
$E \rightarrow E \uparrow E$	{Print('\uparrow')}
$E \rightarrow (E)$	{ }
$E \rightarrow id$	{Print('id')}

The reason for having only print statements for operators and identifier in semantic rules of SDT is that when an infix expression is converted to postfix, do not perform an arithmetic operation, rather display or print the postfix equivalent.

The order in which the semantic actions are taken depend upon the precedence and associativity of operators in a particular language.

5.1.4 Implementing L-Attributed SDD's

Q9. List the various ways in which L-attributed SDD's can be implemented. Also explain the implementation of L-attributed SDD using LL parser.

Answer :

L-attributed grammar can be implemented using,

1. Recursive descent parser
2. Top down parser (LL parser)
3. Bottom up parser (LR (parser).

Implementation of L-attributed SDD's using LL Parser

The steps for implementing L-attributed SDD's using LL parser is as follows,

1. Construct SDD for the given grammar G.
2. Convert the SDD into SDT by removing left recursion.
3. Construct a syntax tree for the grammar with evaluation order.

Example

Consider the following attribute grammar,

$$E \rightarrow E_1 + T$$

$$E \rightarrow E_1 - T$$

$$E \rightarrow T$$

$$T \rightarrow (E)$$

$$T \rightarrow \text{digit}$$

Step 1

The SDD for given grammar is,

Grammar Production	Semantic Rules
$E \rightarrow E_1 + T$	$\{E.val = E_1.val + T.val\}$
$E \rightarrow E_1 - T$	$\{E.val = E_1.val - T.val\}$
$E \rightarrow T$	$\{E.val = T.val\}$
$T \rightarrow (E)$	$\{T.val = E.val\}$
$T \rightarrow \text{digit}$	$\{T.val = \text{digit.lexical value}\}$

Step 2

Since the above SDD is left recursive. The translation scheme for above SDD can be constructed by removing left recursion from it.

The new grammar is,

$$E \rightarrow T, E'$$

$$E' \rightarrow + TE_1$$

$$E' \rightarrow - TE_1$$

$$E' \rightarrow \epsilon$$

$$T \rightarrow (E)$$

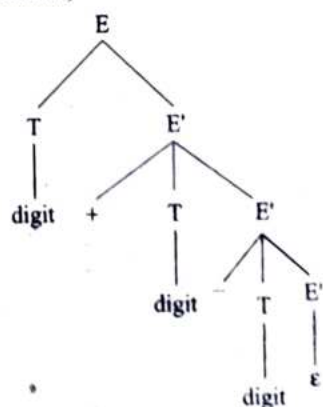
$$T \rightarrow \text{digit}$$

The translation scheme for left recursive grammar is,

Grammar Production	Semantic Rule
$E \rightarrow TE'$	$\{E'.in = T.val\}$ $\{E.val = E'.s\}$
$E' \rightarrow + TE_1$	$\{E'.in = P.in + T.val\}$ $\{E'.S = E_1.S\}$
$E' \rightarrow - TE_1$	$\{E'.in = E.in - T.val\}$ $\{E.S = E_1.S\}$
$E' \rightarrow \epsilon$	$\{E'.S = E'.S\}$
$T \rightarrow (E)$	$\{T.val = E.val\}$
$T \rightarrow \text{digit}$	$\{T.val = \text{digit.lexical value}\}$

Step 3

The syntax tree (or) annotated parser tree for the above grammar is,



The evaluation order depends on the precedence and associativity of arithmetic operators.

LL Parsing for an L-attributed Grammar

An L-attributed grammar can be implemented with LL parser. When bottom up parsing is used with L-attributed grammar the parsers need the following,

1. Stack parser
2. Data items
3. Stack record
4. Action record
5. Synthesized records.

1. **Stack Parser**

It holds the actions.

2. **Data Items**

They (certain data items) are needed to hold copies of attributes for the purpose of evaluation.

3. **Stack Record**

It is capable of holding both terminal and non-terminal symbols of given input string.

4. **Action Record**

It specifies the actions that are to be executed.

5. **Synthesized Attribute**

It is capable of holding the synthesized attribute of non-terminals.

The attributes on stack can be managed based on the following properties.

- Place the non-terminals (T) of an inherited attribute in stack record.
- Evaluate the inherited attribute based on the code present in the action record which is just above the stack record.
- Place the synthesized attribute of non-terminal (T) in synthesized record which is just below the stack record.

Example

Consider the following production.

$$A \rightarrow BC$$

The parsing starts by placing A on TOP of stack and later by replacing it with BC . Consider that the non-terminal C has an inherited attribute say C_i . Then in such situations the evaluation of C_i depends both on the inherited attributes of A as well as on inherited attributes of B . The processing of both A and B can be done by storing the temporary copies of the inherited attributes in action record that evaluates C_i . Similarly the temporary storage is needed by synthesized records to store the synthesized attribute of non-terminals A , B and C .

Q10. Explain how translation is done during recursive descent parsing.**Answer :****Translation During Recursive-Descent Parsing**

Recursive descent parser is a method used for performing translation during parsing. This parser uses a function (F) for every individual non-terminal (F). The function (F) is parameterized with inherited attribute of F and upon invocation it returns the set of synthesized attribute of F .

During the invocation of function F , it is necessary to parse as well as handle the attributes. This is done in the following manner.

- Make a decision regarding the production to be used for expanding the function F .
- Ensure that every terminal is present on the input whenever required.
- Maintain the information about the attribute values in local variables. These values are required for computing the inherited or synthesized attribute.
- Invoke the function associated with the respective non-terminal with the body of the selected production by parameterizing the function with the required arguments.

Q11. Write a short note on bottom up evaluation of inherited attributes.**Answer :****Bottom Up Evaluation of Inherited Attributes**

The inherited attributes are handled by L-attributed definitions. The bottom up parser evaluates the inherited attributes by reducing the right side of the production.

For example, the production $S \rightarrow ABC$ is reduced by removing C , B , A from the parser stack.

The parser stack is carried out as a combination. A pair of state and value are used to implement the parser stack.

State[i] – Holds the grammar symbol A

Value [i] – Holds the synthesized attribute A -a

For example,

Consider the grammar for declaring type of variables.

$$D \rightarrow TL$$

$$T \rightarrow \text{int}/\text{char}$$

$$L \rightarrow L_1, \text{id}/\text{id}$$

The translation scheme for the above grammar is given below,

Productions	Semantic Rules
$D \rightarrow TL$	{L.in L = T.type}
$T \rightarrow \text{int}$	{T.type = int}
$T \rightarrow \text{char}$	{T.type = char}
$L \rightarrow L_1, \text{id}$	{ $L_i = L.in$ {Enter_type(id.entry, L.in)}}
$L \rightarrow \text{id}$	{Enter_type(id.entry, L.in)}

For example consider, the input string char x, y.
z. The bottom up evaluation of inherited attributes is shown below.

Input	State	Production
char x, y, z	char	
x, y, z	T	$T \rightarrow \text{char}$
x, y, z	TL	
y, z	TL	$L \rightarrow \text{identifier}$
y, z	TL	
y, z	TL, y	
z	TL	$L \rightarrow L_1$ identifier
z	TL	
z	TL, z	$L \rightarrow L_1$ identifier
	TL	
	S	$S \rightarrow TL$

The following observations can be made from the above implementation.

1. The decision about the value of L in is always taken by the A.type.
2. The T always resides below the L in the parser stack.
3. The position of T in the value array of parser stack is always known.
4. The enter_type() function copies the corresponding type to the identifier. It takes two parameters.
 - (a) Identifier entry
 - (b) Identifier type

Now, the contents of parser stack become.

1. When $T \rightarrow \text{char}$ the contents of parser stack are shown in figure (1).

* State	Value
T	Char

Figure (1): Parser Stack

2. When $L \rightarrow L$, the contents of parser stack and state stack are shown in figure (2).

State	Value
TL, y	

Top	y
Top-1	,
Top-2	L
Top-3	T

Parser Stack

State Stack

Figure (2)

3. When $L \rightarrow id$ the contents of parser stack and value stack are shown in figure (3).

State	Value
T x	

Top	x
Top-1	char

Parser Stack

Value stack

Figure (3)

Here, the (top-3) value i.e., char is assigned to top element i.e., thus y, will have a char data type.

Thus, it can be said that the bottom up evaluation of inherited attributes is done by copying T.type to L.in with the help of enter_type function.

To translation scheme for grammar of declaring type of variables is as follows.

Production	Code
$S \rightarrow TL$	
$T \rightarrow \text{int}$	Value [top] := int
$T \rightarrow \text{char}$	Value [top] := char
$L \rightarrow L, id$	Enter_type (value[top], value [top-3])
$L \rightarrow id$	Enter_type (value [top], value [top-1])

5.2 INTERMEDIATE-CODE GENERATION

5.2.1 Variants of Syntax Trees

Q12. Explain how a DAG can be drawn for an expression. Give examples.

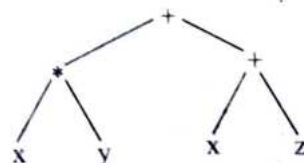
Answer :

A DAG for an expression is a special form of syntax tree for an expression. A DAG for an expression reduces the number of appearances of common or reoccurring sub-expressions and makes the common sub-expression to appear only once.

In a syntax tree a leaf node with a same name can appear any number of times, if it appears those many times in the given expression. However, in a DAG a leaf will appear only once, but will not change the meaning of the expression. This can be explained by taking an example expression,

$$x * y + x + z$$

The syntax tree would be,



Syntax Tree for the Expression $x * y + x + z$

Now, a DAG can be,



As x is appearing twice in the syntax tree, a DAG for that syntax tree will have only one appearance of x , but the edges to x from $*$ and $+$ remains the same.

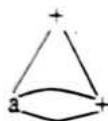
Example 1

Expression is $(a + a + a)$

Syntax tree can be,



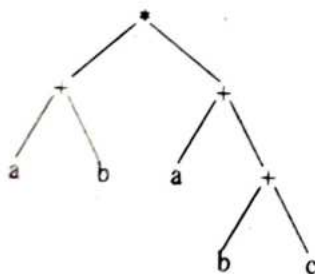
Equivalent DAG can be,



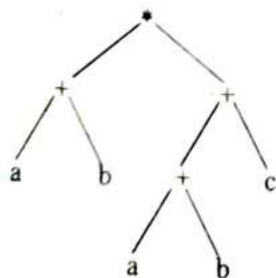
Example 2

Expression is $(a + b) * (a + b + c)$

Syntax trees can be,



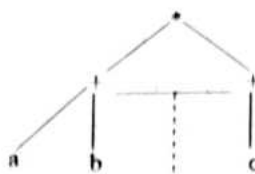
Syntax Tree(i) for the Expression



Syntax Tree(ii) for the Expression

In syntax tree (i), a, b are appearing twice. So, the DAG for the syntax tree can be,

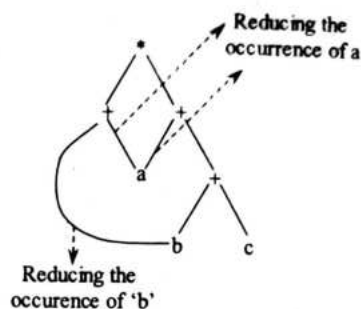
DAG for Syntax Tree (i)



This arrow from left '+' to right '+' means the result of $(a + b)$ is the left child of right '+'

Thus, in this DAG the number of occurrences of a, b and the sub expression $a + b$ are reduced.

DAG for Syntax Tree (ii)



Q13. Discuss the value-number method for constructing DAG's.

Answer :

The value-number method for constructing DAG's can be explained as follows,

Consider an expression $a = a * 6$, whose corresponding DAG or syntax tree is shown below,

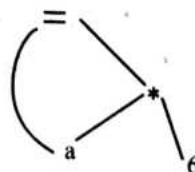


Figure (1): DAG for $a = a * 6$

An equivalent array representation for the above DAG looks as follows,

Entry for 'a'		
1	id	
2	num	6
3	*	1 3
4	=	1 5
5	---	---
6	-	---

Figure (2): Array Representation

As shown in figure (2), the nodes corresponding to a DAG are implemented as an array of records where each record in the array represents the node of a DAG. In the array, each record contains more than one field, where the first field represents an operation code, which holds the label for that node. The leaves of node contains one more field indicating the lexical value which can be a symbol table pointer a constant. The interior nodes of a DAG contains two more fields, which holds left child and right child.

In array representation, the nodes of a DAG are referred by integer index of a record that corresponds to that node in the array. Here, integer index is known as 'value-number' of that node. For example, the node labelled '+' has value number '3', whose left child contains value number '1' and right child contains value-number '3'. Similarly, the node labelled '=' has value number '4' whose left child contains value number '1' and right child contains value-number '5'.

The nodes of DAG stored in an array are referred by its value-number (integer index of a node). The signature of a node represented by triple $\langle op, l, r \rangle$.

Where,

$op \rightarrow$ label of the node

$l \rightarrow$ value-number of its left child

$r \rightarrow$ value-number of its right child

The signature of a node is used to check-whether the signature that is similar is present in the array or not if same signature exists, then it returns the value-number otherwise, creates a new node with same signature (i.e., $\langle op, l, r \rangle$) And returns its value-number.

5.2.2 Three - Address Code

Q14. What is a three address code? What are its types?

Answer :

Three Address Code

Three address code is a sequence of three address statements where in two address are used for storing operands and one address is used for storing result.

That is the three-address code converts each statement in the program into a sequence of statements which should have at least two operands (so two addresses are used here) and at most three operands. The three-address code can have statements of the form

$$a := b \text{ operator } c$$

Figure: General Format of 3-address Code

Here, a, b, c can be constants, programmer defined variables or compiler generated temporary storages. Operator can be arithmetic, logical or boolean.

Hence, it is clear for the general format of three address code that,

- They can be maximum of three address for a, b and c.
- They exist only single operator in a three-address statement.

$\therefore$ For an expression $a * b + c$, the three address could be,

$$\text{temp1} = a * b$$

$$\text{temp2} = \text{temp1} + c$$

Where temp1 and temp2 are temporary storages that are generated by compiler.

Three-address code represents the syntax tree. For example consider the syntax tree for input $4 + a * b$.

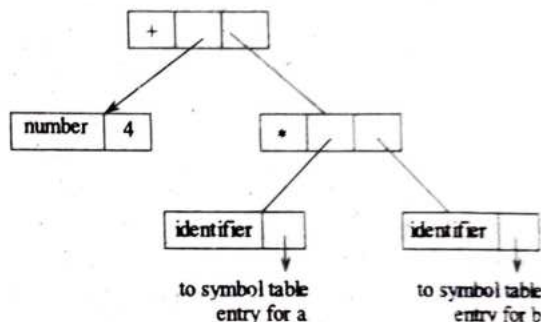


Figure: Syntax Tree for $4 + a * b$

Three-address code for above syntax tree is represented as,

$$\text{temp1} := a * b$$

$$\text{temp2} := 4 + \text{temp1}.$$

Where temp1 and temp2 are temporarily variables used by the computer.

Three Address Statements

Three-address statements used commonly are,

- An assignment statement like

$$r := a \text{ OP } b$$

is a tree-address statement where OP is any binary operator such as arithmetic or logical operator.
- An assignment statement like,

$$r := \text{OP } a$$

is a three-address statement where OP is a unary operator such as logical negation, shift operator or unary minus.
- An assignment statement like,

$$a := b$$

which assigns the value of b to a.
- An unconditional jump statement like,

$$\text{goto } S$$

Which causes the flow of control to reach a three-address statement labeled S and execute it as the next statement.

5. A conditional jump statement like,
if a relop b goto S .
Here, relop is one of the relational operators
i.e., $>, <, \geq, \leq, =, \neq$. If condition is true,
this statement executes the statement labeled S ,
otherwise executes the statement that follows it.
6. A procedure call like call p, m with param a and
return i . Here, p is a procedure name, m is the
number of parameters and i is a return value from
procedure which is optional.
7. Index assignment statements like,
 $a[i] := b$ and $a := b[i]$
Where a, b and i are the data objects.
8. Statements that assign address and printers like
 $a := \&b$ and $a := *b$.

Q15. Explain different types of representations of three address codes.

Answer :

Types of Representation of Three Address Code

Three-address code statements can be represented as record containing the fields for operator and operands. The following are the methods for implementing three address code.

1. Quadruples
2. Triples
3. Indirect triples.

1. Quadruples

In quadruple, a record contains four fields namely operator, operand1, operand2 and result. Where operand1 and operand2 are two operands for an operator. An operator field holds internal representation for the operator. The result field contains the result of their operation.

For the three-address statement of the form $a := b \text{ OP } c$, OP is placed in operator field, b and c are placed in operand1 and operand2 and the result field contains a .

Statements with unary operator do not require the operand2. The operand2 and result field is not used by the param operators, conditional and unconditional jump statements place the label in the result field.

Example

The table below shows the quadruples for the assignment statement $r := a * -b + a * -b$.

$$\begin{aligned} t_1 &:= -b \\ t_2 &:= a * t_1 \\ t_3 &:= -b \\ t_4 &:= a * t_3 \\ t_5 &:= t_2 + t_4 \\ r &:= t_5 \end{aligned}$$

Operator	Operand 1	Operand 2	Result
Uminus	b		t_1
*	a	t_1	t_2
Uminus	b		t_3
*	a	t_3	t_4
+	t_2	t_4	t_5
	t_5		r

Table (a): Quadruples for Example Statement

The fields operand1, operand2 and result contains the pointers to the symbol table entries for their name. Therefore, wherever the temporary names are created, they must be entered into the symbol table.

2. Triples

This method overcomes an overhead associated with quadruples that is entering temporary names into the symbol table by using the triples that computes a temporary value to refer it. Doing so, reduces the number of fields of a record to three fields namely operator, operand1 and operand2. Here operand1 and operand2 contains,

- (i) The pointer to the symbol table, if the names are programmer defined or constants.
- (ii) The pointer into the triple structure itself if these are temporary values.

An entry like (1) in operand columns refers to the statement number.

The table below shows the triples for the assignment statement.

$$r := a * -b + a * -b.$$

Triple	Operator	Operand 1	Operand 2
(0)	Uminus	b	
(1)	*	a	(0)
(2)	Uminus	b	
(3)	*	a	(2)
(4)	+	(1)	(3)
(5)	assign	r	(4)

Table (b): Triples for Example Statement

3. Indirect Triples

Indirect triples structure does not list the triples like triples structure. Instead, it maintains a table of triples and represents the statements by a pointer to triples in the desired order. An advantage of indirect triples is that, when the statements move around as they often do, user just needs to reorder the statement list. The table below shows indirect triples for the assignment statement,

$$r := a * -b - b + a * -b.$$

Statement	Triple
(0)	(10)
(1)	(14)
(2)	(11)
(3)	(13)
(4)	(12)
(5)	(15)

Triple	Operator	Operand1	Operand2
(10)	Uminus	b	
(11)	*	a	(10)
(12)	Uminus	b	
(13)	*	a	(11)
(14)	+	(14)	(13)
(15)	assign	r	(12)

Table (c): Indirect Triples for Example Statement

Q16. Discuss briefly on static-single assignment form.

Answer :

Static-single assignment form is abbreviated as SSA form or simply SSA. It refers to an intermediate representation (IR) that allows to apply code optimization techniques.

Static single-assignment form is different from three address code in two distinct ways which are as follows.

In SSA, every assignment is made to different variable name (i.e., Unique variable). Thus, the name given as static single assignment. The following figure illustrates differentiation of SSA and three-address code with intermediate program.

$$a = p + q$$

$$a = a - t$$

$$a = b * s$$

$$a = r - a$$

$$b = a + b$$

$$a_1 = p + q$$

$$b_1 = a_1 - t$$

$$a_2 = b_1 * s$$

$$a_3 = r - a_2$$

$$b_2 = a_3 + b_1$$

(1) Three address code (2) Static single-assignment form

If the control flow passes through 'if' statement, then $\phi(i_1, i_2)$ returns the value ' i_1 '. on the other hand, if control-flow passes through 'else' statement then $\phi(i_1, i_2)$ returns value ' i_2 ' ie; the ϕ -function returns the argument value corresponding to the path of the control flow.

Q17 Translate the following expression :

$$-(a + b) * (c + d) + (a + b + c) \text{ into}$$

- (a) Quadruples
(b) Triples
(c) Indirect triples.

Answer :

Given expression is,

$$R = -(a + b) * (c + d) + (a + b + c)$$

Three-address code sequence for the given expression is,

$$t_1 := a + b$$

$$t_2 := -t_1$$

$$t_3 := c + d$$

$$t_4 := t_2 * t_3$$

$$t_5 := t_1 + c$$

$$t_6 := t_4 + t_5$$

$$R := t_6$$

(a) Quadruple

Operator	Operand1	Operand2	Result
+	a	b	t_1
uminus	t_1		t_2
+	c	d	t_3
*	t_2	t_3	t_4
+	t_1	c	t_5
+	t_4	t_5	t_6
:=	t_6		R

(b) Triple

Triple No.	Operator	Operand1	Operand2
(0)	+	a	
(1)	+	b	(0)
(2)	uminus	(1)	
(3)	+	c	
(4)	+	d	(3)
(5)	*	(2)	(4)
(6)	+	(1)	(3)
(7)	+	(5)	(6)
(8)	assign	R	(7)

(c) Indirect Triple

Statement	Triple No.
(0)	(10)
(1)	(11)
(2)	(12)
(3)	(13)
(4)	(14)
(5)	(15)
(6)	(16)
(7)	(17)
(8)	(18)

Triple No.	Operator	Operand1	Operand2
(10)	+	<i>a</i>	
(11)	+	<i>b</i>	(10)
(12)	uminus	(11)	
(13)	+	<i>c</i>	
(14)	+	<i>d</i>	(13)
(15)	*	(12)	(14)
(16)	+	(11)	(13)
(17)	+	(15)	(16)
(18)	assign	<i>R</i>	(17)

5.3 RUN-TIME ENVIRONMENT

5.3.1 Stack Allocation of Space

Q18. Write about stack allocation strategy. Explain in detail about activation tree and activation record.

Answer :

Stack Allocation Strategy

Static allocation strategy allocates the memory for all data objects at compile time. The program variables are bound to the storage once it is allocated to them. As the binding of variables for storage is done at compile time which don't change at run time. Therefore, a run time support package is not required. This strategy implements static binding. In static binding, each time a procedure is called the variables are bound to the same storage locations.

This property retains the values of variables across various activations of a procedure. That is, when the control returns to a procedure, the local variables hold the same values they had when the control left the procedure last time.

(a) Activation Tree

An activation tree is used to determine the way the control flows between procedures. During program execution, the control flow is sequential among procedures. In a procedure, the execution begins at the starting point of its body. At the end of the procedure, control is returned to the calling instruction.

Each execution of the procedure is called an activation. Its lifetime is the sequential instructions between the first and last steps in execution of a procedure, the time that is spent on executing other procedures called by it, the time spent by other procedures in calling some other procedures and so on.

Simply, the activation tree depicts how control enters and leaves activations in an activation tree,

- (i) The root implies the activation of main() program.
- (ii) The node implies an activation of a procedure.
- (iii) A node c is a child of a node p if the control flow is from activation p to activation c .
- (iv) A node p is left to node q only if lifetime of p occurs before lifetime of q .

Example

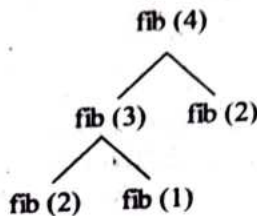


Figure: An Activation Tree for the Procedure Call fib (4)

Here, the activation tree is a graph of function calls. At any point during execution the activation stack, where the activation records are maintained, describes some path starting from root of activation tree to the desired node.

(b) Activation Record

Activation record or frame is a contiguous block of storage that contains all the information required for a single execution of a procedure. An activation record consists of seven fields as shown in the figure.

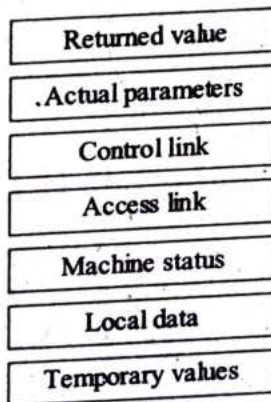


Figure: An Activation Record

- (i) **Temporary Values**
This field holds all the temporary values that are used in the evaluation of expressions.
- (ii) **Local Data**
This field holds all the data that is local to a procedure.
- (iii) **Machine Status**
This field saves the status of the machine when a call to procedure occurs. The machine status includes the values of program counter and registers which must be restored when the control returns from the call.
- (iv) **Access Link**
This is an optional field. This field holds a link to data (non local) held in other activation records.
- (v) **Control Link**
This is also an optional field that contains a link to caller's activation record.

(vi) **Actual Parameters**

This field contains the actual parameters that are used by the calling procedure to pass them to the called procedure. In practice, these parameters are passed in registers for greater efficiency.

(vii) **Returned Value**

This field contains a return value that is used by the called procedure to return to the calling procedure. Again, this value is returned in a **register for greater efficiency**.

When a procedure is called, the size of each of these fields can be known. The sizes of various fields can be determined at compile time. There is an exception if a procedure contains an array whose size can be determined the value of actual parameters which is known only at the run time when the procedure is called.

Q19. With a neat diagram explain the division of task between caller and callee.

Answer :

Procedural calls are often performed through a technique called calling sequence, which includes a special code for allocating an activation record on the stack and feeding data into its fields. This code in the calling sequence is segmented or divided among the caller (the calling procedure) and the callee (the called procedure).

Following figure depicts the division of tasks performed by the caller and callee.

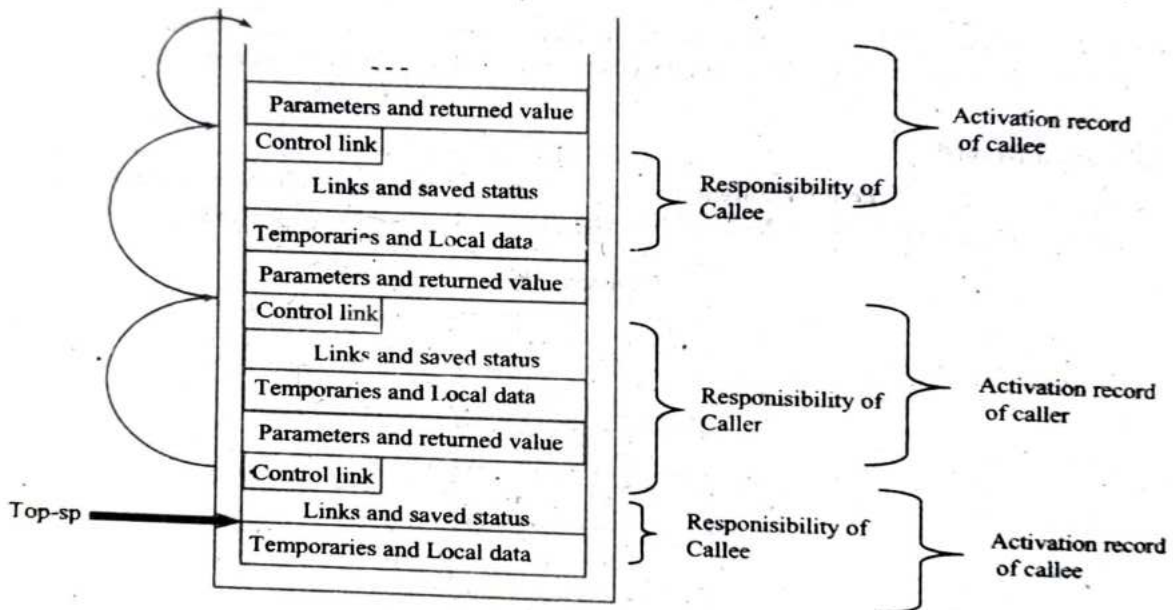


Figure: Division of Tasks Performed by Caller and Callee

The sequence of steps performed by the caller and the callee in a procedural call are as follows.

Step 1

The process starts-off with the evaluation of the actual parameters by the caller, followed by their pushing onto the stack.

Step 2

In this step, the caller executes a 'call' assembly instruction which pushes and stores the return address and the previous value of top-sp onto the stack in callee's activation record. This is followed by the increment status fields of the callee.

Step 3

In this step, the callee, saves the information of such as register value and other status. Followed by pushing of top-sp to the top of the stack.

Step 4

In this step, the callee generates a local space on the stack and initializes the local data, followed by the start of execution.

A suitable, specific return sequence for the above process would be.

Step 1

In this step, the callee restores the status of the machine to the initial state by placing the return value next to the parameter.

Step 2

In this step, the callee restores the top-sp register and other registers by utilizing the information present in machine-status field. And then branches to the return address that was provided by the caller in the status field.

Step 3

In this step, the caller traces the return value by utilizing the present value of the registers top-sp and then copies it to its own status field.

Furthermore, these calling sequences might permit the arguments to differ in number from call to call. And an essential entity in the whole process are the target codes of the caller and the callee. A caller target code is aware of the number and types of arguments, it is providing to the callee. Therefore, the caller knows the size of the parameter area. But, in case of a callee's target code, there is no information about the parameter area. This is because, the callee's target code must be made ready to manage the other calls which prohibits it from examining the parameter field before it is called.

Q20. Explain the method of handling fixed length data and variable length data.

Answer :

The data such as variables used in the source program will be either fixed length or variable length. However fixed-length and variable-length data is handled as follows,

Consider a procedure (A) containing a fixed-length data calls as well as variable-length data calls another procedure (B). In this case, the data on the stack is organized as shown in figure below.

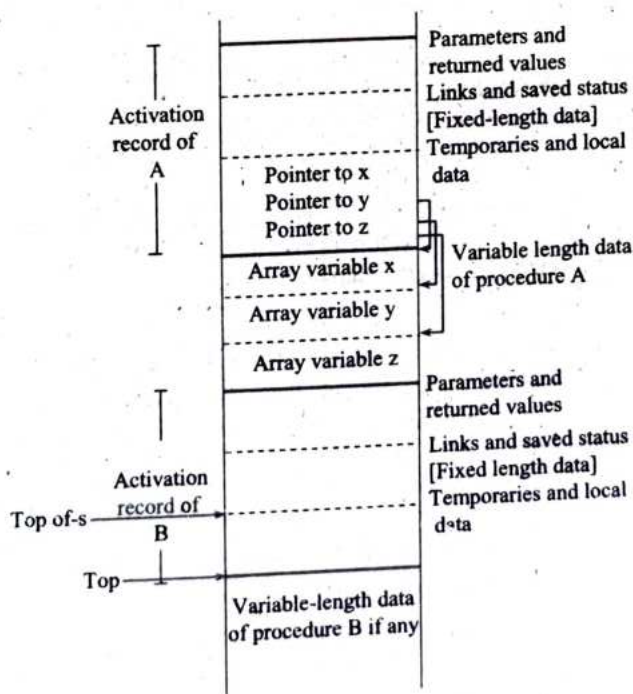


Figure: Handling of Fixed and Variable-length Data

Fixed length data is mostly placed in the middle part of the activation record (AR), such as control link, access link and saved machine status. A pointer (top-of-s) is made to point to the end of these fields.

This helps in evaluating the fixed offset of these fields allowing the access data present in these fields. Apart from this, most of the local variables also have fixed-length and are placed at the end of the activation record. However, the same pointer is used to evaluate fixed offset.

Variable length data is placed above the top of the stack (top) instead of placing it in the activation record. However, this data is handled with help of the top of stack pointer which points to actual top of the stack. This pointer points to the location where an activation record begins.

The activation record of procedure 'A' contains pointer to the variable length data (away variables) placed outside the AR so as to access them when the procedure 'A' gets executed.

Repositioning of the stack pointer top and top-of-s is performed during compile time.

5.3.2 Access to Nonlocal Data on the Stack

Q21. Explain how allocation strategies can access non-local variable.

Answer :

A non-local variable is a variable which is not locally defined in a procedure.

The Scope Rules of a Language

The **lexical-scope** or **static-scope** rule determines the declaration of a name based on the program text alone. Examples of languages that use this scope rule are Pascal, C, Ada and ML.

The **dynamic-scope** rule determines the declaration of a name at run time-based on the current activation records. The languages Lisp, APL and Snobol use this scope rule.

Data Access without Nested Procedure

'C' does not allow nested procedure declarations, therefore the lexical-scope rules for C are simple. In a C program the declaration of variables is followed by procedure definitions (procedures in C are called functions). A function is said to have a reference to a non local name x iff x is declared outside any function. The scope of x is within the functions that follow its declaration. Therefore x can be redeclared within the function.

The stack allocation strategy used for local names can also be used for lexically-scoped languages that does not allow nested procedures. A name declared outside any procedure is a non local storage for all non local names memory is allocated statically at compile time. As the position of storage, is known to user, the static address of that name is used to access its value.

All other names are local to a procedure whose storage is within the current activation record which is at the top of the stack and accessible through the *top* pointer. This scheme does not work with nested procedures because an access to a non local may refer to data deep in the stack.

An advantage of static allocation for non locals is that, procedures can be passed as parameters and returned as results. In the absence of nested procedures a name which is non local to one procedure is non local to all procedures. Therefore, any procedures regardless of how is it activated can use static address of that non local. When a procedure is returned as a result, the non locals of that procedure can be accessed similarly because they are bound to store allocated memory statically for them.

Nested Procedures Issues

If the nested procedure declarations are allowed by a language and normal static scoping rule is used then, accessing of data becomes more problematic. Suppose, the declaration of 'a' is suddenly nested with 'b' at compile time. Then, about the locations of their activation records are not known to users at runtime. Actually, either a/b or both can be recursive. So, there may be a possibility that various activation records of either a or b exist on the stack.

Nested Procedure Declaration using ML

ML is the most effective language that can be used for nested procedures. The semantics and syntax of these languages are as follows,

1. ML is defined as a functional language in which variables initialized and declared once cannot be changed. However, in few cases such as array elements, they can be changed with the help of special function calls.

2. The variables can be declared and initialized by using the following statement.

`val <N> = <exp>`

Here, 'N' represents name of the variable

'exp' represents expression.

The function can be declared by using the following syntax.

`fun <N> (<arg>) = <Bd>`

Here, 'N' represents name of the function

'arg' represents argument of the function

'Bd' represents body of the function

4. The let statements are used for function bodies and can be declared by using the following syntax.

`let <LOD> in <stmts> end`

Here,

'LOD' represents list of definitions

'stmts' represents statements.

Nesting Depth

If the procedures are not nesting inside any other procedure then, these procedures can be named as "nesting depth 1". For instance, each and every C function is at nesting depth '1'. Eventually, at nesting depth 'i' if a procedure 'a' is declared suddenly within a procedure then, nesting depth 'i + 1' is assigned to procedure 'a'.

Program

`fun sort(inFile, OutFile) =`

`let`

`Val V = array (11, 0);`

`fun readArray(inFile) = ---- V ----;`

`fun exchange (c, d) = ---- V ----;`

`fun quicksort(p, q) =`

`let`

`Val V1 = ----;`

`fun partition (r, s) = ---- V ---- V1 ---- exchange ----`

`in`

`end`

`in`

`---- V ---- readArray ---- quicksort ----`

`end`

`in`

`---- V ---- readArray ---- quicksort ----`

`end;`

The above program is the version of a quicksort in ML style which uses nested functions. Initially, the nesting depth is 1 and then, it is changed to 3.

Implementation of Lexical-scope for Nested Procedures

In lexically-scoped languages with nested procedures like ML, the storage for non local names would belong to activation records of different procedures. There are two ways to find the activation record that contains the storage bound to non local names. They are,

1. Access link
2. Displays.

1. Access Link

In this method a pointer called an access link is added to each activation record. If procedure p_1 is nested immediately within p_2 then the access link in p_1 's activation records points to the access link in p_2 's activation record which is the most recent activation of p_2 .

The figures below show the run time stack during an execution of the sort program.

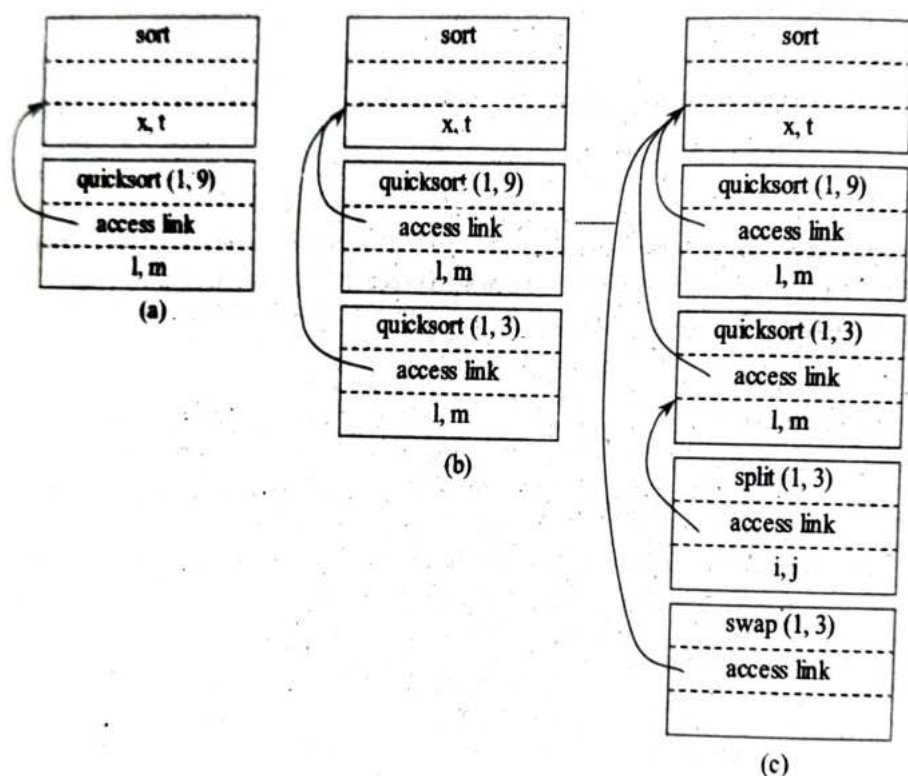


Figure (1): Run Time Stack During an Execution of the Sort Program

Since *sort* is the main program having no enclosing procedure; the access link for its activation record is empty (figure 1(a)). For each activation of quick sort its accesslink points to the access link of *sort*. (figure 1(b)). Figure 1(c) shows that the access link in *split* points to the access link in the record of the most activation record of *quicksort*.

Suppose that a procedure p having nesting depth $n(p)$ references a non local name x with hashing depth $n(x)$ such that $n(x) \leq n(p)$ then the storage bound to non local x is found as follows.

- (a) When p is a current procedure its activation record will be at the top of the stack. Then, follow $n(p) - h(x)$ number of access links from the current record in the stack. The value of $n(p) - n(x)$ can be computed at compile time and can follow a link by performing a single indirection operation.

- (b) By following $n(p) - n(x)$ lines an activation record for the procedure where x is declared that is where x is local to that procedure. Once a procedure containing x is declared, can be reached the storage bound to it is a fixed offset relative to a position in the activation record. This offset is also relative to the access link.

Hence, compute the storage for non local x at compile time which is given by the following pair and store it in the symbol table. The pair is,

$(n(p) - n(x), x \text{ offset within the activation record})$ the same lexical-scope rules apply to a nested procedure passed as a parameter.

Displays

This method maintains an array d of pointers to activation records called a *display*. An array element $d[i]$ points to the activation containing the storage for nonlocal x with nesting depth i .

Suppose the top of the stack contains an activation record of a procedure p at nesting depth j . Then the display elements from $d[0]$ to $d[j-1]$ contain a pointer to the most recent activations of the procedures that lexically enclose p and the $d[j]$ element contains a pointer to the activation record of p .

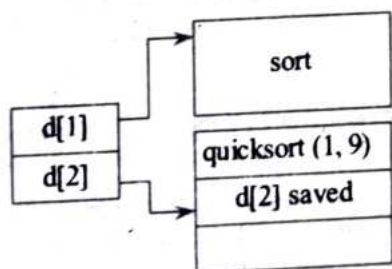
Display provides faster access to non locals than accesslinks. It needs to access display element and then follow a single pointer.

The following changes to display occurs when an activation record for a procedure at nesting depth i is created:

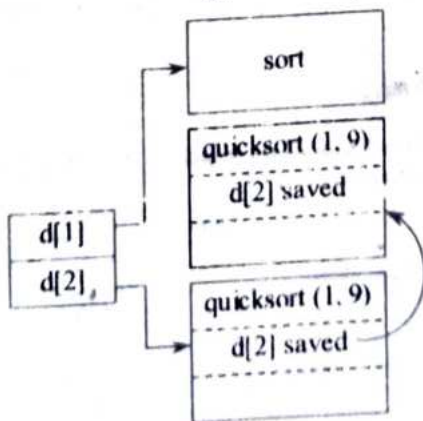
- The value of $d[i]$ element of display is saved in the new activation record.
- Display element $d[i]$ is set to point to the new activation record.

Reset the value of $d[i]$ to the saved value just before an activation ends.

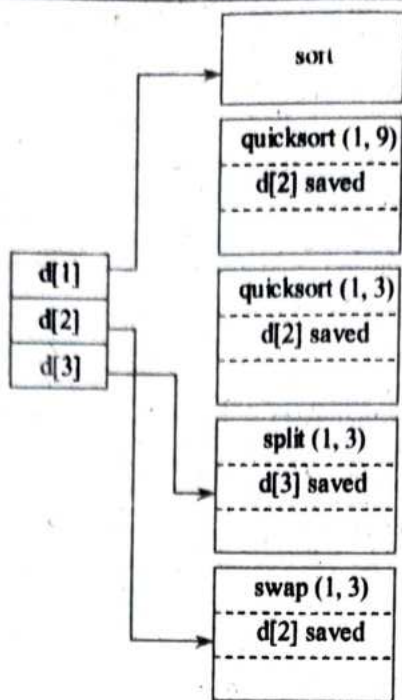
The figure (a), (b) and (c) below show the run time stack and display during an execution of the sort program.



(a)



(b)



(c)

Figure (2): Run-time Stack

When a procedure having nesting depth of j calls, a procedure at depth i then there are two cases depending on whether the called procedure is nested within the calling procedure.

Case (i): $j < i$

If this is the case then it means $i = j + 1$ that is, the called procedure (i) is nested within the calling procedure (j). Therefore there are no changes in the first j elements of the display only user need to set the display element $d[i]$ to the new activation record. This case is shown in figure 2(a) when sort calls the procedure quick sort.

Case (ii): $j \geq i$

The enclosing procedures of the called and calling procedure must be at same nesting depths i.e., 1, 2, ..., $i - 1$.

In this case, the old value of $d[i]$ element of the display is saved in the new activation record and $d[i]$ element is set to point to the new activation record. Since, the first $i - 1$ elements are not changed the display maintains the correct values. This case is shown in figure 2(b) where $i = j = 2$. This condition is possible because quick sort is called recursively. Another example of this case is shown in figure 2(c). Here, the procedure split (1, 3) at nesting depth 3 calls swap (1, 3) at nesting depth 2. The nesting depth of their enclosing procedure sort is same i.e., $i = j = 1$.

5.3.3 Heap Management

Q22. What are the two basic functions performed by memory manager? Also list the properties desired for a memory manager.

OR

Explain about Heap management.

Answer :

The two basic functions of memory manager in heap storage includes,

1. Memory allocation
2. Memory deallocation.

1. Memory Allocation

In this the memory manager maintains a contiguous chunk of free space of memory in heap storage. This memory is allocated to the program which has made a memory request for a variable or object. If the allocated size of variable is available in heap then allocation is done directly. Otherwise, the needed size is made available by increasing storage space. This can be done by getting sequential bytes of virtual memory from operating system. If the space is completely filled up, then this information (about no free space is available) is redirected to the application program by the memory manager.

2. Memory Deallocation

In this the memory manager returns the deallocated space back to the pool of free space. This free space get mixed up with the already available free space in heap. This deallocated memory can later be reused for other allocation requests.

Properties of Memory Manager

Properties of memory manager are as follows,

(i) Maintaining Space Efficiency

Space efficiency can be obtained by reducing the fragmentation of the program. To achieve this, the memory manager minimizes the amount of total heap space required by the program. Due to which a larger program can run on fixed virtual address space.

(ii) Maintaining Program Efficiency

Program efficiency can be achieved if the memory manager efficiently utilizes the available subsystem memory. This utilization helps to run the program at a faster rate.

(iii) Minimizing Over Head

The overhead can be minimized if allocation and deallocation operation are performed efficiently. As these are the frequent operations performed by memory manager.

VERY SHORT QUESTIONS WITH SOLUTIONS (VSQS)**Q1. Define Syntax-Directed Translation.****Answer :**

Syntax Directed Translation (SDT) add semantic rules to productions of context free grammar. It defines the evaluation order of semantic rules and hence the abstraction at this level is less.

Q2. What are the different types of attributed grammars?**Answer :**

The different types of attributed grammars are as follows,

1. S-attributed grammar
2. L-attributed grammar.

Q3. Define annotated parse tree.**Answer :**

Annotated parse tree is a specialized tree used to analyze the translation specified by SDD at each node of the parse tree.

Q4. How to find evaluation order for SDD's?**Answer :**

The evaluation order for attribute instance in a given parse tree can be determined with the help of a special type of graph called dependency graph.

Q5. Write down the ways in which L-attributed SDD's can be implemented.**Answer :**

L-attributed grammar can be implemented using,

1. Recursive descent parser
2. Top down parser (LL parser)
3. Bottom up parser (LR (parser).

Q6. What do you mean by three-address code?**Answer :**

Three address code is a sequence of three address statements where in two address are used for storing operands and one address is used for storing result.

Q7. What is activation record?**Answer :**

Activation record or frame is a contiguous block of storage that contains all the information required for a single execution of a procedure.

Q8. List the properties of memory manager.**Answer :**

The properties of memory manager are as follows,

1. Maintaining space efficiency
2. Maintaining program efficiency
3. Minimizing over head.

Q9. What are the two basic functions of memory manager in heap storage?**Answer :**

The two basic functions of memory manager in heap storage are given below,

1. Memory allocation
2. Memory deal location.

FREQUENTLY ASKED QUESTIONS & IMPORTANT QUESTIONS

Q1. What is syntax directed translation? What is syntax directed definition? Give its general form.

Answer :

For answer refer Unit-V, Page No. 136, Q1.

Important Question

Q2. Describe about S-attributed and L-attributed grammars.

Answer :

For answer refer Unit-V, Page No. 136, Q2.

Important Question

Q3. How to determine an evaluation order for the attribute instance in a given parse tree?

Answer :

For answer refer Unit-V, Page No. 139, Q5.

Important Question

Q4. Explain the translation process using suitable example.

Answer :

For answer refer Unit-V, Page No. 142, Q7.

Important Question

Q5. Construct the Syntax Directed Definition to convert infix notation into postfix notation.

Answer :

For answer refer Unit-V, Page No. 143, Q8.

Important Question

Q6. List the various ways in which L-attributed SDD's can be implemented. Also explain the implementation of L-attributed SDD using LL parser.

Answer :

For answer refer Unit-V, Page No. 145, Q9.

Important Question

Q7. Explain how a DAG can be drawn for an expression. Give examples.

Answer :

For answer refer Unit-V, Page No. 147, Q12.

Important Question

Q8. What is a three address code? What are its types?

Answer :

For answer refer Unit-V, Page No. 149, Q14.

Important Question

Q9. Translate the following expression :

$-(a + b) * (c + d) + (a + b + c)$ into

(a) Quadruples

(b) Triples

(c) Indirect triples.

Answer :

For answer refer Unit-V, Page No. 151, Q17.

Important Question